RaiSE Jumpstart — Sesión 1

Ingeniería de Software Confiable con IA

Kurigage Tech Leads · Febrero 2026

El Jumpstart: Nuestra Semana

Sesión	Día	Qué Hacemos
1	**Hoy**	**Entender el problema y la solución**
2	Martes	Evaluar sus repos: factores críticos de éxito
3	Miércoles	Onboarding: configurar proyecto, primera story
4	Jueves	Office hours: preguntas y troubleshooting
5	Viernes	Office hours: autonomía y graduación

Al final de la semana: cada línea de producto tiene su repo configurado con RaiSE y al menos una story entregada.

La Promesa

"IA va a escribir el 80% del código."

"Los desarrolladores van a ser 10x más productivos."

"Solo dile qué hacer y lo construye."

La Realidad

- El código se genera rápido pero se rompe fácil
- Cada sesión empieza de cero — no hay memoria
- El asistente no conoce tus reglas, tu arquitectura, tu contexto
- Merge requests llenos de código que "funciona" pero no cumple estándares
- Tech debt se acumula a velocidad de IA

Pregunta al grupo: ¿Quién ha tenido que reescribir código generado por IA que "pasaba tests" pero no seguía las convenciones del equipo?

El Problema No Es la IA

El problema es que **no hay gobernanza**.

Un junior sin mentoría escribe código funcional pero frágil. Un senior con contexto escribe código que evoluciona.

La IA hoy es el junior más rápido del mundo — sin contexto, sin memoria, sin disciplina.

¿Qué Pasa Sin Gobernanza?

Sin Gobernanza	Con Gobernanza
Cada sesión empieza de cero	Memoria acumulada entre sesiones
"Best practices" genéricas	Tus reglas, tu arquitectura
Genera y reza	Test → Código → Verificar → Commit
No sabes qué decidió ni por qué	Cada decisión es trazable
Velocidad sin dirección	Velocidad con confiabilidad

Lo Que Funciona: Principios de Manufactura Lean

La industria del software no es la primera en resolver este problema.

Toyota lo resolvió hace 70 años.

Jidoka — Parar al Detectar Defectos

En Toyota, cualquier trabajador puede detener la línea de producción al ver un defecto.

> **En software con IA:** El asistente debe parar cuando detecta incoherencia, ambigüedad o violación de reglas — no seguir generando tokens.

La velocidad sin calidad es desperdicio. Generar 500 líneas con un bug arquitectónico es peor que generar 50 líneas correctas.

Kaizen — Mejora Continua

Cada iteración enseña algo. Ese aprendizaje se captura y alimenta la siguiente.

> **En software con IA:** Cada story completada produce patrones. Esos patrones informan las siguientes sesiones. El sistema mejora con el uso.

No por magia — por repetición disciplinada.

Poka-yoke — Diseño a Prueba de Errores

Mecanismos que hacen imposible (no improbable) cometer ciertos errores.

En software con IA:

- Gates de calidad que bloquean avance sin tests
- Branches que se anidan correctamente por diseño
- Scope commits que documentan intención antes de escribir código

Los Humanos Definen, Las Máquinas Ejecutan

Este es el principio central.

Tú decides:

- Qué construir y por qué
- Qué reglas sigue tu código
- Qué estándares son no negociables
- Cuándo algo está "done"

La IA ejecuta:

- Dentro de tus reglas
- Con memoria de lo aprendido
- Siguiendo flujos de trabajo estructurados
- Parando cuando algo no cuadra

RaiSE: La Implementación

Estos principios no son teoría — son un framework que pueden instalar hoy.

La Tríada

Rai (Partner IA — Ejecución + Memoria)

gobernado por

Tú (Estrategia, Juicio, Ownership)

colabora con

RaiSE (Metodología + Toolkit)

- **Tú** decides *qué* y *por qué*
- **Rai** ejecuta con memoria y juicio calibrado
- **RaiSE** provee la disciplina que hace la colaboración confiable

Ninguno está completo sin los otros. Sin la metodología, la IA genera sin dirección. Sin la IA, la metodología es burocracia. Sin ti, no hay juicio.

Los Cuatro Pilares

1. Gobernanza

Tu proyecto tiene reglas. RaiSE las hace explícitas y ejecutables.

Constitución

Principios no negociables ("Type annotations en todo el código")

Guardrails

Reglas que tu código debe seguir (MUST vs SHOULD)

PRD

Qué estás construyendo y por qué

Arquitectura

Cómo está organizado tu sistema

La IA lee estos documentos al inicio de cada sesión. No son aspiracionales — son operativos.

2. Skills

Flujos de trabajo estructurados que reemplazan el "dile qué hacer y reza".

01

/rai-story-start

Scope: ¿qué estamos construyendo?

02

/rai-story-design

Spec: ¿cómo va a funcionar?

03

/rai-story-plan

Tareas: ¿cuáles son los pasos?

04

/rai-story-implement

Construir: test, código, verificar, commit

05

/rai-story-review

Reflexionar: ¿qué aprendimos?

06

/rai-story-close

Merge: limpiar y entregar

Cada paso produce un artefacto. Cada artefacto alimenta el siguiente. Trazabilidad completa.

3. Memoria

Lo que hace a RaiSE diferente de cualquier otro workflow.

Patrones

Aprendizajes que se acumulan entre sesiones

Calibración

Datos de velocidad: estimado vs real

Sesiones

Historial de trabajo con contexto de continuidad

Knowledge Graph

Mapa unificado de gobernanza + código + aprendizajes

Mañana tu IA sabe lo que aprendió hoy. La semana que viene, lo que aprendió esta semana.

4. Knowledge Graph

El mapa que conecta todo:

- **Gobernanza** → qué reglas existen y por qué
- **Arquitectura** → cómo está organizado el código
- **Patrones** → qué hemos aprendido
- **Calibración** → qué tan rápido trabajamos

Un comando: `rai memory build`. Toda la información disponible, consultable, cargable en contexto.

¿Cómo Se Ve en Práctica?

Inicio de Sesión

```
rai session start --project . --context
```

Tu IA recibe:

- Tu perfil de desarrollador
- Estado de la sesión anterior
- Patrones relevantes de memoria
- Contexto de lo que estás construyendo

~150 tokens. Conciencia completa.

El Ritmo de Trabajo

Cada feature sigue el mismo ciclo:

Scope
Documentar qué sí y qué no

Design
Decisiones de integración

Plan
Tareas atómicas con estimados

Build
TDD: test → código → verificar → commit

Review
¿Qué aprendimos? → Memoria

Merge
Limpiar y entregar

El ritmo se vuelve natural después de 2-3 stories. Empiecen con algo pequeño (XS o S).

Fin de Sesión

```
rai session close --summary "Lo que logré" --type feature --project .
```

Captura:

- Resumen de lo logrado
- Patrones aprendidos
- Estado para la siguiente sesión
- Contexto de continuidad

Mañana, la sesión empieza donde quedó hoy.

Lo Que RaiSE NO Es

- **No es "vibe coding con pasos extra"** — Es disciplina que produce velocidad
- **No reemplaza desarrolladores** — Los hace más efectivos
- **No es rígido** — El framework evoluciona con tu equipo (Kaizen)
- **No requiere cambiar tu CI/CD** — Se integra a tus herramientas existentes
- **No es magia** — Es repetición disciplinada que compone aprendizaje

Para Mañana: Instalar y Explorar

Paso 1: Instalar

```
pip install rai-cli
rai --version
```

Paso 2: Inicializar en un repo de prueba

```
cd mi-repo-de-prueba
rai init
```

Paso 3: Explorar lo que se creó

```
ls -la .raise/
ls -la governance/
```

Paso 4: Primera sesión

```
rai session start --name "Tu Nombre" --project .
```

No intenten hacer todo. Solo instalar, inicializar, y mirar qué se creó. Mañana lo exploramos juntos.

Documentación

docs.raiseframework.ai

Getting Started

Instalación y primera sesión

Conceptos

Memoria, Skills, Gobernanza, Knowledge Graph

Guías

Tu primera story, configurar un proyecto

CLI Reference

Todos los comandos y flags

Todo disponible en español e inglés.

¿Preguntas?

"Raise your craft, one story at a time."

Resumen

1. **El problema**
   no es la IA — es la falta de gobernanza

2. **Principios Lean**
   (Jidoka, Kaizen, Poka-yoke) aplicados a software con IA

3. **RaiSE**
   implementa esos principios: Gobernanza + Skills + Memoria + Knowledge Graph

4. **La Tríada**
   Tú (juicio) + Rai (ejecución) + RaiSE (disciplina)

5. **Para mañana**
   instalar, inicializar, explorar